Project 1 — REST API with PHP/MySQL + Nginx

Student: Landon Patton

Course: ASE 230

Date: October 2025

This presentation documents the design, implementation, testing, and deployment of my REST API project using PHP, MySQL, and Nginx.

Slide 2 — Project Overview

- REST API implemented in PHP
- Data stored in MySQL
- Deployed with Nginx under WSL on Windows 11
- Tested via cURL and HTML/JS front-end
- Fully authenticated CRUD API

Slide 3 — Project Folder Structure

```
project1-restapi/
    code/
        db/connect.php
        public/index.php
        public/index.html
        src/auth.php
        tests/curl_tests.sh
    presentation/
        tutorial.md
    .env
```

Slide 4 — Database Schema (Users)

```sql
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  email VARCHAR(255),
  role ENUM('user','admin') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Screenshot: `presentation/screenshots/db_users.png`

Slide 5 — Database Schema (Items & Tokens)

Items Table

```
CREATE TABLE items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  owner_id INT NOT NULL,
  title VARCHAR(255),
  description TEXT,
  price DECIMAL(10,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tokens Table

```
CREATE TABLE tokens (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user id INT NOT NULL,
```

Slide 6 — API Endpoint List

1. `POST /users` – register
2. `POST /login` – authenticate user
3. `GET /items` – list items
4. `GET /items/{id}` – view one
5. `POST /items` – create (auth required)
6. `PUT /items/{id}` – update (auth required)
7. `DELETE /items/{id}` – delete (auth required)
8. `GET /search?q=` – search items
9. `GET /users/{id}` – get user (auth required)

Slide 7 — Authentication Process

- User registers via `/users`
- Logs in via `/login`
- Server issues a random 64-char token
- Token stored in `tokens` table with expiry
- All protected endpoints require:

```
Authorization: Bearer <token>
```

Screenshot: `presentation/screenshots/db_token.png`

Slide 8 — Register Endpoint

Request:

```
curl -X POST http://127.0.0.1/users \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"pw1234"}'
```

Response:

```
{"id":"10"}
```

Slide 9 — Login Endpoint

Request:

```
curl -X POST http://127.0.0.1/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"pw1234"}'
```

Response:

```
{"token":"b42517..."}
```

Slide 10 — Create Item (Protected)

Request:

```
curl -X POST http://127.0.0.1/items \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"title":"Book","description":"Fiction","price":5.99}'
```

Response:

```
{"id":"1"}
```

Slide 11 — List & View Items

Request:

```
curl http://127.0.0.1/items
```

Response:

```
[{"id":1,"title":"Book","price":"5.99"}]
```

Request:

```
curl http://127.0.0.1/items/1
```

Slide 12 — Update & Delete Items

Update:

```
curl -X PUT http://127.0.0.1/items/1 \
-H "Authorization: Bearer $TOKEN" \
-d '{"title":"Updated Book","price":7.50}'
```

Delete:

```
curl -X DELETE http://127.0.0.1/items/1 \
-H "Authorization: Bearer $TOKEN"
```

Slide 13 — Search Endpoint

Example:

```
curl "http://127.0.0.1/search?q=Updated"
```

Response:

```
[{"id":1,"title":"Updated Book","price":"7.50"}]
```

Slide 14 — cURL Test Script

File: `code/tests/curl_tests.sh`

- Registers a new user
- Logs in, stores token
- Creates, reads, updates, deletes items
- Tests invalid token rejection

Screenshot: `presentation/screenshots/curl_tests.png`

Slide 15 — HTML/JS Client

File: `code/public/index.html`

- Register, Login, and Create Item forms
- Uses `fetch()` to call the API
- Displays results in `<pre>` blocks

Screenshot: `presentation/screenshots/client.png`

Slide 16 — Nginx Configuration

File: `/etc/nginx/sites-available/project1`

```
server {
    listen 80;
    server_name localhost;
    root /home/lando/projects/project1-restapi/code/public;
    index index.php index.html;

    location / {
        try_files $uri /index.php$is_args$args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
}
```

Slide 17 — Nginx Deployment

Commands used:

```
sudo apt install nginx php-fpm
sudo ln -s /etc/nginx/sites-available/project1 /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

Screenshot: `presentation/screenshots/nginx_status.png`

Slide 18 — My Errors & Fixes

Port 80 already in use

- Apache2 was running and using port 80, so I had to stop it.

Slide 19 — GitHub & Verification

Uploaded folders:

- code/ – API source
- presentation/ – Slides & screenshots
- .env excluded (added to .gitignore)
  Final test:

```
./code/tests/curl_tests.sh
```

Slide 20 — Lessons Learned

- Learned how to use PHP with MySQL through PDO
- Understood REST routing logic and tokens
- Nginx setup was challenging but rewarding
- Debugging improved my Linux confidence
- Next steps: explore Docker deployment